LIBMEM : MODE D'EMPLOI

La bibliothèque `libmem` permet de tester des algorithmes de remplacement de pages.
L'arborescence est la suivante :

```
libmem
├── Makefile
├── README
├── TME
│   ├── Fifo.c
│   ├── Fifo.h
│   ├── LRU.c
│   ├── LRU.h
│   ├── Makefile
│   ├── bench
│   ├── mainFifo.c
│   └── mainLRU.c
├── algorithms
│   ├── LFU.c
│   ├── Random.c
│   ├── mainLFU.c
│   └── mainRandom.c
├── bench
├── bin
├── include
│   ├── LFU.h
│   ├── Random.h
│   ├── Swapper.h
│   └── libmem.h
├── lib
├── obj
└── src
    ├── Swapper.c
    └── libmem.c
```

Le répertoire `src` contient les sources de la bibliothèque
Le répertoire `include` contient les fichiers d'en-tête de libmem (`libmem.h, Swapper.h`) et les fichiers propres aux exemples (`LFU.h, Random.h`).

Le répertoire `algorithms` contient un exemple de main (`main.c`) ainsi que des exemples d'algorithmes de remplacement de pages :
`LFU.c` implémente l'algorithme Least Frequently Used
`Ramdon.c` implémente un algorithme qui choisit une page aléatoirement

Le répertoire `TME`, les fichiers à compléter pour le TME

1. FONCTIONS DE LA LIBRAIRIE

Remarque : dans le code de la libmem, la terminologie anglaise est utilisée : frame = case en mémoire physique, page = page en mémoire virtuelle.

Structure de données

La libmem maintient une structure de données « Swapper » définie dans Swapper.h :

```
typedef int Page;

typedef struct Swapper {
…
      unsigned int frame_nb;        /* Nombre de cases de la mémoire physique */
      Page *      frame;            /* Tableau des cases en mémoire */
      void *      private_data;    /* Donnée privée propre à chaque stratégie */
} Swapper;
```

Le champ `frame` indique pour chaque case la page correspondante :
Pour la case i, si i est libre frame[i] = -1, sinon frame[i] = numéro de la page.

Fonctions

libmem fournit deux fonctions :

```
#include "Swapper.h"

int initSwapper(
      Swapper* swap,
      unsigned int nc,                                  //Logical number of frames
      int (*Init)(Swapper*),                            //Init for private data
      void (*Reference)(Swapper*,unsigned int),         //Reference to keep stats
      unsigned int (*Choose)(Swapper*),                 //Choose function
      void (*Finalize)(Swapper*)                        //Finalize function
);
```

initSwapper permet d'initialiser la structure swap avec nc cases. 4 fonctions sont passées en paramètre. `Init` sera appelée à l'initialisation, `Reference` à chaque accès à une case, `Choose` à chaque défaut de page et `Finalize` à la terminaison.

Pour programmer une stratégie de remplacement de pages appelée par exemple « MaStrategie », il faut donc :

1) programmer les 4 fonctions suivantes :

```
int initMaStrategie(Swapper *swap);
```

Où est la variable swap est celle initialisée par la fonction initSwapper.
Cette fonction permet éventuellement d'allouer et d'initialiser le champ `private_data` de la struture swap. La fonction doit retourner 0 en cas de succès

```
void referenceMaStrategie(Swapper *swap, unsigned int frame) ;
```

Fonction qui sera appelée lors d'un accès à la case mémoire numéro `frame`.

```
unsigned int chooseMaStrategie(Swapper *swap) ;
```

Fonction qui sera appelée lors d'un défaut de page. Elle doit retourner le numéro de la case contenant la page victime.

```
Void finalizeMaStrategie(Swapper *swap) ;
```

Fonction appelée à la fin du programme pour libérer de qui a été allouée par initMaStratégie.

2) Appeler initSwapper de la manière suivante :

```
Swapper *s,
initSwapper(&s, nbcases, initMaStrategie, refenceMaStrategie, chooseMaStrategie,
finalizeMaStrategie);
```

```
#include "libmem.h"

int   swapSimulation(Swapper *swap,FILE *f);
```

Lance la simulation des accès aux pages. `swap` doit être préalablement initialisé par initSwapper. Le fichier `f` contient un entier par ligne : la première ligne indique le nombre de cases de la mémoire physique et les autres lignes, la suite de référence mémoire.

2. EXEMPLE

L'exemple suivant illustre l'utilisation des primitives. On souhaite implémenter une stratégie de remplacement de page de type LFU (Least Frequently Used). Pour cela, il faut avoir un tableau qui compte le nombre de références à une case. En cas de remplacement de page, il suffit de choisir la case ayant le compteur avec une valeur minimum.

include/LFU.h :

```
#include "Swapper.h"

int initLFUSwapper(Swapper*,unsigned int);
```

algorithms/LFU.c :

```
#include "LFU.h"
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>

int   initLFU(Swapper*);
void  referenceLFU(Swapper*,unsigned int frame);
unsigned int chooseLFU(Swapper*);
void  finalizeLFU(Swapper*);


int initLFUSwapper(Swapper*swap,unsigned int frames){
      initSwapper(swap,frames,initLFU,referenceLFU,chooseLFU,finalizeLFU);
}

int   initLFU(Swapper*swap){

      /* Allouer un tableau avec un compteur d'utilisation pour chaque case */
```

```
	swap->private_data = calloc(swap->frame_nb,sizeof(int));
	int * use = (int*)swap->private_data;
	int i;

	/* Initialisation du tableau */

	for( i=0 ; i<swap->frame_nb ; i++ )
		use[i] = 0;
	return 0;
}

void  referenceLFU(Swapper*swap,unsigned int frame){
	int i;
	int * use = (int*)swap->private_data;

	/* A chaque acces à la case frame augmente son compteur d'utilisation */
	use[frame]++;
}


unsigned int chooseLFU(Swapper*swap){
	int i, frame = 0;
	int * use = swap->private_data;

/* Choisir la case (contenant une page)  ayant le plus petit compteur */

	for ( i=0 ; i<swap->frame_nb ; i++ ){
		if( swap->frame[i] == -1 ){
			frame = i;
			break;
		}
		if( use[i] < use[frame] )
			frame = i;
	}

	use[frame] = 0;

	return frame;
}

void  finalizeLFU(Swapper*swap){
	free(swap->private_data);
}
```

algorithms/main.c :

```
#include "libmem.h"
#include "LFU.h"

int main(int argc,char*argv[]){
	unsigned int frame_nb;
	Swapper     s;

	scanf("%i",&frame_nb);

	/* Initialisation du Swapper de la stratégie LFU */
	initLFUSwapper(&s,frame_nb);

	/* Lancer la simulation */
	if(swapSimulation(&s,stdin)<0){
		printf("Error during swap simulation !!!\n");
		return -1;
	}

	return 0;
}
```

3. UTILISATION

La `libmem` est disponible sous forme d'archive libmem.tgz sur le moodle de l'UE.

Tout d'abord, copiez la libmem dans un répertoire, puis décompressez l'archive :

```
$ tar xvfz libmem.tgz
```

Le sous-répertoire `libmem/algorithms` contient l'exemple (LFU.c) précédent ainsi qu'une élection aléatoire (Random.c). Dans `libmem` un fichier makefile permet de générer directement l'exécutable et la bibliothèque en exécutant les commandes suivantes :

```
$ cd libmem
$ make
```

Pour créer une nouvelle stratégie (MaStrategie), il faut : créer un fichier MaStrategie.h dans le répertoire include, un fichier MaStrategie.c dans algoritms et modifier le fichier main.c dans algorithms.

Le fichier `libmem/bench` contient un exemple de suite de références mémoire pour 3 cases. Pour lancer la libmem sur cet exemple il faut entrer :

```
$ bin/mainLFU < bench
Page 1 referenced
LFU uses:

/!\ PAGE FAULT !!! /!\
Frame 0 has been choosen
(frame 0: 1) (frame 1: _) (frame 2: _)

Page 2 referenced
LFU uses: (page:1 time:1)

/!\ PAGE FAULT !!! /!\
Frame 1 has been choosen
(frame 0: 1) (frame 1: 2) (frame 2: _)

Page 3 referenced
LFU uses: (page:1 time:1) (page:2 time:1)

/!\ PAGE FAULT !!! /!\
Frame 2 has been choosen
(frame 0: 1) (frame 1: 2) (frame 2: 3)

Page 1 referenced
(frame 0: 1) (frame 1: 2) (frame 2: 3)

Page 2 referenced
(frame 0: 1) (frame 1: 2) (frame 2: 3)

Page 4 referenced
LFU uses: (page:1 time:2) (page:2 time:2) (page:3 time:0)

/!\ PAGE FAULT !!! /!\
Frame 2 has been choosen
(frame 0: 1) (frame 1: 2) (frame 2: 4)

Page 1 referenced
(frame 0: 1) (frame 1: 2) (frame 2: 4)

Page 2 referenced
(frame 0: 1) (frame 1: 2) (frame 2: 4)
```

```
Page 4 referenced
(frame 0: 1) (frame 1: 2) (frame 2: 4)

4/9 ~ 44.444443%
```

Par défaut la libmem est en mode « verbeux », pour désactiver le mode, il faut commenter dans le makefile la ligne :

```
CFLAGS=-D_DEBUG_
```

Merci à Mathieu Valéro pour avoir développé la libmem
Pour tout commentaire, ou rapport de " bug " : Pierre.Sens@lip6.fr